Actividad: organiza las ventanillas de una oficina

Versión con datos reales y *bootstrap*

Enunciado

Una oficina recibe clientes entre las 7:00 y las 22:00 y dispone de un máximo de cinco ventanillas. Cada ventanilla abierta necesita un trabajador y puede atender a un único cliente a la vez.

Tu objetivo es programar en Python un algoritmo genético que decida cuántas ventanillas abrir cada hora. Debes equilibrar el tiempo de espera de los clientes y el tiempo de inactividad de los trabajadores.

A diferencia de otras versiones de este problema, **no se te da la distribución de las llegadas**. Dispones del registro histórico de la oficina y debes construir las jornadas simuladas a partir de él mediante *bootstrap*. Además, la afluencia no es igual todos los días de la semana, así que la asignación óptima de ventanillas puede cambiar de un día a otro.

Datos del problema

El día se divide en quince franjas de una hora, de 7:00–8:00 a 21:00–22:00, numeradas de 1 a 15. Todos los clientes necesitan 10 minutos de atención. El tiempo se mide en minutos desde las 7:00 y puede tomar valores decimales, de modo que la jornada de apertura es el intervalo $[0, 900)$.

El conjunto de datos

Los datos están en el archivo `llegadas_oficina.csv`. Recogen las llegadas de 67 días laborables: todos los comprendidos entre el 2 de febrero y el 30 de abril de 2026, salvo los festivos de Semana Santa, y además los lunes, martes y miércoles de las semanas del 4 y del 11 de mayo. Por eso no todos los días de la semana tienen el mismo número de observaciones:

	lunes	martes	miércoles	jueves	viernes
Días registrados	14	15	15	12	11

Cada fila del archivo es un cliente:

`fecha`	`dia_semana`	`hora_llegada`
`2026-02-02`	`lunes`	`07:28:38`
`2026-02-02`	`lunes`	`07:51:55`
`2026-02-02`	`lunes`	`08:29:03`
	⋮	

- `fecha`: día de la observación, en formato `AAAA-MM-DD`.
- `dia_semana`: `lunes`, `martes`, `miércoles`, `jueves` o `viernes`.
- `hora_llegada`: hora de llegada del cliente con precisión de segundos (`HH:MM:SS`).

Las filas de cada día están ordenadas por hora de llegada. Como la precisión es de un segundo, pueden aparecer clientes que llegan exactamente a la vez.

Análisis exploratorio previo

Antes de programar el algoritmo genético, carga el archivo y:

1. Convierte cada `hora_llegada` a minutos desde las 7:00 (por ejemplo, `07:28:38` $\rightarrow 28 + 38/60 \approx 28{,}633$).
2. Calcula, para cada día de la semana, el número medio de llegadas en cada franja horaria y represéntalo en un diagrama de barras (un panel o una serie por día).
3. Describe brevemente el perfil: ¿cuándo están las horas punta? ¿Qué días difieren más del resto? ¿Cuánto varía el número total de clientes entre días del mismo tipo?

Representación de una solución

Cada individuo es una lista de quince números enteros entre 1 y 5, uno por franja. Por ejemplo:

$$[2, 2, 3, 3, 3, 3, 3, 3, 3, 4, 5, 5, 5, 4, 3]$$

Esta solución abre dos ventanillas de 7:00 a 9:00, tres de 9:00 a 16:00, cuatro de 16:00 a 17:00, y así sucesivamente.

Reglas de atención

1. Hay una cola única. Los clientes se atienden por orden de llegada; si llegan a la vez, se utiliza su orden en los datos.
2. Las ventanillas se numeran del 1 al 5. Cuando se necesitan k ventanillas, pueden recibir clientes las ventanillas 1 a k. Si hay varias libres, se elige la de menor número.
3. La cola se conserva al cambiar de hora. Si una ventanilla debe cerrar, termina la atención en curso y después cierra; no recibe nuevos clientes mientras no vuelva a estar habilitada.
4. A las 22:00 no llegan más clientes. Las ventanillas de la última franja siguen atendiendo hasta vaciar la cola y terminar todos los servicios.

Generación de jornadas mediante *bootstrap*

En lugar de suponer un modelo para las llegadas, se generan a partir de la **distribución empírica** de los tiempos entre llegadas observados. Como la afluencia depende de la hora y del día de la semana, se construye una distribución empírica distinta para cada pareja (día de la semana, franja).

Muestra de tiempos entre llegadas

Sea D el día de la semana que se quiere planificar (por ejemplo, `viernes`) y h una franja, que cubre el intervalo $[60(h-1),\, 60h)$. Para cada fecha histórica de tipo D, sean $t_1 \le t_2 \le \cdots \le t_k$ sus llegadas dentro de la franja. Sus tiempos entre llegadas son

$$t_1 - 60(h-1), \quad t_2 - t_1, \quad \ldots, \quad t_k - t_{k-1}.$$

El primer salto se mide desde el inicio de la franja. El tiempo que transcurre entre la última llegada y el final de la franja *no* se incluye, porque no se llega a observar la siguiente llegada.

Uniendo los saltos de todas las fechas de tipo D se obtiene la muestra $x_1, \ldots, x_n$ de la franja h, que se ordena: $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$.

La ECDF y su inversa

La función de distribución empírica (ECDF) de la muestra es

$$\widehat{F}_n(x) = \frac{1}{n}\,\#\{\, i : x_i \le x \,\},$$

una función escalonada que salta $1/n$ en cada observación. Su inversa generalizada es

$$\widehat{F}_n^{-1}(u) = \inf\{\, x : \widehat{F}_n(x) \ge u \,\} = x_{(\lceil nu \rceil)}, \qquad 0 < u \le 1.$$

Por el método de la transformada inversa, si $U \sim \mathcal{U}(0,1)$, entonces $X = \widehat{F}_n^{-1}(U)$ sigue la distribución empírica: toma cada valor $x_{(i)}$ con probabilidad $1/n$. Por tanto, generar así los saltos es exactamente un *bootstrap* (remuestreo con reemplazo de los tiempos observados).

Como `rng.random()` devuelve valores en $[0,1)$, en el código se usa la versión equivalente $x_{(\lfloor nu \rfloor + 1)}$.

Simulación de las llegadas

Para simular cada franja, parte de su hora de inicio y suma sucesivos saltos generados con $\widehat{F}_n^{-1}$ de esa franja. Registra solo las llegadas anteriores al final de la franja. Si el siguiente salto alcanza o supera ese final, descártalo y empieza en la siguiente franja con su propia distribución empírica. No añadas una llegada automática al cambiar de hora.

Código proporcionado

El Listado 1 contiene el archivo `bootstrap.py` completo. Úsalo tal cual en tu entrega: `generar_jornadas(df, dia)` devuelve una lista con las $M = 100$ jornadas del día indicado, cada una como un array con los instantes de llegada en minutos desde las 7:00.

Listado 1: `bootstrap.py`: generación de jornadas con la ECDF inversa.

```python
"""bootstrap.py -- Generación de jornadas por bootstrap con la ECDF inversa."""
import numpy as np
import pandas as pd

N_FRANJAS = 15          # franjas de una hora entre las 7:00 y las 22:00
DURACION = 60.0         # minutos por franja
M = 100                 # jornadas simuladas
DIAS = ["lunes", "martes", "miércoles", "jueves", "viernes"]


def cargar_llegadas(ruta="llegadas_oficina.csv"):
    """Lee el CSV y añade la columna t: minutos (decimales) desde las 7:00."""
    df = pd.read_csv(ruta)
    hms = df["hora_llegada"].str.split(":", expand=True).astype(int)
    df["t"] = (hms[0] - 7) * 60 + hms[1] + hms[2] / 60
    return df


def tiempos_entre_llegadas(df, dia):
    """Para un día de la semana, devuelve una lista de 15 arrays ordenados:
    los tiempos entre llegadas observados en cada franja (todas las fechas)."""
    sub = df[df["dia_semana"] == dia]
    muestras = [[] for _ in range(N_FRANJAS)]
    for _, g in sub.groupby("fecha", sort=True):
        t = g["t"].to_numpy()
        for h in range(N_FRANJAS):
            inicio = h * DURACION
            th = t[(t >= inicio) & (t < inicio + DURACION)]
```

```
            if len(th) > 0:
                # primer salto: desde el inicio de la franja hasta la 1.ª llegada
                saltos = np.diff(np.concatenate(([inicio], th)))
                muestras[h].extend(saltos)
    return [np.sort(np.array(m)) for m in muestras]


def ecdf_inversa(x_ordenado, u):
    """Inversa de la ECDF: para 0 <= u < 1 devuelve x_(k+1), con k = floor(n*u).
    Cada observación se elige con probabilidad 1/n (remuestreo con reemplazo)."""
    n = len(x_ordenado)
    return x_ordenado[int(n * u)]


def generar_jornada(muestras, rng):
    """Simula las llegadas de una jornada sumando saltos generados con la ECDF
    inversa de cada franja. Descarta el salto que alcanza o supera el final."""
    llegadas = []
    for h in range(N_FRANJAS):
        if len(muestras[h]) == 0:
            continue
        t = h * DURACION
        fin = t + DURACION
        while True:
            t += ecdf_inversa(muestras[h], rng.random())
            if t >= fin:
                break
            llegadas.append(t)
    return np.array(llegadas)


def generar_jornadas(df, dia, semilla=42, m=M):
    """Genera las m jornadas de un día de la semana con un generador propio."""
    muestras = tiempos_entre_llegadas(df, dia)
    rng = np.random.default_rng(semilla)
    return [generar_jornada(muestras, rng) for _ in range(m)]
```

Reproducibilidad. Para cada día de la semana se crea un generador con `np.random.default_rng(42)` y se consume un número uniforme por cada salto, recorriendo las jornadas y las franjas en orden. No vectorices ni reordenes estas llamadas: cambiaría la secuencia aleatoria y dejaríais de obtener todos exactamente las mismas jornadas.

Evaluación mediante Monte Carlo

El algoritmo genético busca una asignación de ventanillas que funcione bien ante distintas jornadas. Monte Carlo estima el coste esperado de cada individuo simulando las $M = 100$ jornadas obtenidas por *bootstrap*.

Para un individuo x y una jornada d, calcula:

- $E_d(x)$: suma de los minutos de espera de todos los clientes, desde su llegada hasta el inicio de su atención. No incluye los 10 minutos de servicio.
- $P_d(x)$: suma del tiempo de inactividad de las ventanillas habilitadas entre las 7:00 y las 22:00. Una ventanilla acumula inactividad cuando está habilitada y no atiende a nadie.

El coste de la jornada y la estimación que se debe minimizar son:

$$C_d(x) = 2E_d(x) + P_d(x), \qquad \widehat{C}(x) = \frac{1}{100}\sum_{d=1}^{100} C_d(x).$$

Un minuto de espera de un cliente pesa el doble que un minuto de inactividad de un trabajador. La espera se contabiliza también después de las 22:00 si hay clientes pendientes; la inactividad solo se contabiliza antes de esa hora.

Genera las 100 jornadas una sola vez, antes de iniciar la optimización. Evalúa todos los individuos y generaciones sobre esas mismas 100 jornadas. Así se comparan las asignaciones bajo las mismas condiciones. Usa un generador aleatorio independiente para las operaciones del algoritmo genético.

Simulación de cada jornada

Simula en tiempo continuo: no redondees los tiempos a minutos enteros. Los eventos son llegadas, finalizaciones de atención y cambios de franja.

1. Comienza a las 7:00 con la cola vacía y todas las ventanillas libres.
2. Avanza al siguiente evento. Acumula la inactividad del intervalo transcurrido multiplicando su duración, dentro de $[0, 900)$, por el número de ventanillas habilitadas que estaban libres.
3. Si coinciden eventos, finaliza primero las atenciones, actualiza después la franja e incorpora las llegadas. A continuación, asigna clientes de la cola a las ventanillas habilitadas libres.
4. Una atención iniciada en t termina en $t + 10$. Continúa hasta alcanzar las 22:00 y haber terminado todas las atenciones.

Funcionamiento del algoritmo genético

1. **Población inicial.** Genera 20 individuos aleatorios con quince genes entre 1 y 5.
2. **Evaluación.** Calcula $\widehat{C}(x)$ mediante las 100 jornadas de Monte Carlo.
3. **Torneo.** Elige dos individuos distintos al azar y conserva el de menor coste medio estimado. Resuelve los empates al azar.
4. **Selección.** Realiza dos torneos para obtener los dos padres. Un mismo individuo puede ganar ambos torneos.
5. **Cruce.** Elige un punto de corte entre las posiciones 1 y 14 e intercambia las partes finales para crear dos hijos.
6. **Mutación.** Cada gen de cada hijo tiene un 5 % de probabilidad de sustituirse por otro entero distinto entre 1 y 5.
7. **Nueva generación.** Conserva una copia del mejor individuo sin modificarla y añade hijos hasta completar 20 individuos. Si queda un hueco, añade solo uno de los dos hijos.
8. **Finalización.** Repite durante 100 generaciones y muestra la solución con menor coste medio estimado. No es obligatorio alcanzar el óptimo.

El algoritmo se ejecuta **una vez para cada día de la semana**, cada vez con sus propias 100 jornadas *bootstrap*.

Consejo de rendimiento. Como las jornadas son fijas, el coste de un individuo no cambia entre generaciones. Puedes guardar en un diccionario los costes ya calculados (con la tupla de genes como clave) para no simular dos veces el mismo individuo.

Organización del código

Organiza el programa en los siguientes archivos:

`fuse.py`
Define los parámetros, lee el día de la semana, genera las 100 jornadas y ejecuta el algoritmo genético.

`bootstrap.py`
Se proporciona en el Listado 1. Lee los datos, construye la muestra de tiempos entre llegadas de cada franja y genera las jornadas con la ECDF inversa.

`simulacion.py`
Simula la atención de una jornada y evalúa cada individuo mediante Monte Carlo.

`torneo.py`
Implementa el torneo entre dos individuos y devuelve el ganador.

`seleccion.py`
Selecciona los dos padres utilizando la función de torneo.

`cruce.py`
Implementa el cruce de un punto y devuelve los dos hijos.

`mutacion.py`
Implementa la mutación de los genes con una probabilidad del 5 %.

El programa debe ejecutarse indicando el día de la semana, por ejemplo:

```
python fuse.py viernes
```

Si se ejecuta `python fuse.py` sin argumentos, debe resolver los cinco días consecutivamente.

Resultados y entrega

Para cada día de la semana, muestra las ventanillas abiertas en cada franja, el coste medio estimado, la espera total media por jornada, la inactividad media por jornada y el número medio de clientes por jornada. Incluye la espera media por cliente, calculada como

$$\frac{\sum_d E_d}{\sum_d N_d},$$

donde N_d es el número de clientes de la jornada d; si no hay clientes, toma este valor como cero.

Resume además las cinco soluciones en una tabla (filas: días de la semana; columnas: franjas) y compáralas con el perfil de llegadas obtenido en el análisis exploratorio.

Entrega los archivos de Python, el gráfico del análisis exploratorio y responde brevemente:

1. ¿Qué función cumple el algoritmo genético, cuál cumple Monte Carlo y cuál cumple el *bootstrap*?
2. ¿Por qué se utilizan las mismas jornadas para comparar los individuos?
3. ¿Qué efecto tendría aumentar el número de jornadas simuladas? ¿Mejora eso la información disponible sobre la oficina? Ten en cuenta que solo hay entre 11 y 15 días históricos de cada tipo.
4. ¿Por qué generar $X = \widehat{F}_n^{-1}(U)$ con U uniforme equivale a remuestrear con reemplazo? ¿Qué ventajas e inconvenientes tiene frente a suponer que los tiempos entre llegadas son exponenciales? ¿Puede la simulación producir un salto que no se haya observado nunca?

5. Compara el número medio de llegadas por franja de las jornadas simuladas con el de los datos. ¿Por qué la simulación genera algo más de clientes? *Pista: ¿qué saltos no llegan nunca a la muestra?*
6. Los saltos se generan de forma independiente, mezclando observaciones de fechas distintas. ¿Qué estructura de los datos se pierde? Piensa, por ejemplo, en días que están más cargados que otros durante toda la jornada.
7. ¿Por qué cambia la solución de un día de la semana a otro? ¿Qué ocurriría si mezclaras todos los días en un único conjunto de remuestreo?
8. Los jueves y viernes tienen menos días registrados que el resto. ¿Cómo afecta eso a la fiabilidad de sus soluciones? ¿Qué podrías hacer para comprobarlo?